

Prompt History

Billing Explainability — MCP Server on Cloudflare Workers

Candidate: Avanti Ketkar

Role: Senior Director of Engineering, Billing Platform — Cloudflare

Assignment: Optional AI application build (LLM · workflow/coordination · chat input · memory/state)

Build window: 27–30 August 2026 — designed, built, deployed and documented

How to read this

Three sections. **Design prompts** are the six prompts that decided what got built, reproduced verbatim with the decision each produced. **What running it exposed** is the six defect classes found once the thing actually ran. **What deploying it exposed** is the three found when it left my laptop. Each defect carries its symptom, its real cause, the fix, and what it changed about how I was working.

Routine iteration is left out: restarts, typos, port confusion, "what was that flag again". Keeping those would pad the log without telling a reviewer anything. What is here is every defect that changed the design or the method.

On completeness. Scoping conversations preceded Prompt 1 in an earlier session and are not reproduced verbatim, because I do not have those transcripts. Their conclusions are summarised below and marked as summary, not as prompt text.

Where the build stood at Prompt 1 (summary, not prompt text)

- **Domain:** billing explainability — *why did this account's bill change, and what is the evidence?*
- **Why:** it is the target team's charter in miniature (self-serve through enterprise, quote-to-cash) and a domain where correctness and auditability are non-negotiable, which is what makes the AI problem interesting rather than decorative.
- **Platform:** Workers, Workers AI, Durable Objects for session state and the audit trail, Workflows for coordination.
- **Open:** the shape. A chat application was the obvious build and the wrong one.

Design decisions

Prompt	Decision	Reasoning
1	MCP server on the Agents SDK, not a chat wrapper	A chat wrapper shows a model can be called. An MCP server with a real authorization model shows a revenue system can be made safely agent-accessible — the problem the platform faces next.
1	Plus an agent that consumes it, with chat above	Covers the assignment's "chat input" directly rather than by interpretation, and makes the coordination layer real.
2		

	The agent's write surface is its own workspace, never the ledger	12 reads, 4 writes, none of which move money or speak to a customer. Value lives in the join across systems; risk lives in mutation.
2	No agent-callable <i>apply_credit</i> ; approval is out-of-band	An in-band confirmation flag puts the gate in the same trust domain as the thing gated. The control is that the credential to approve is one the model never holds.
2	<i>account_id</i> is a selector, not an authorization claim	Model output must never widen scope. Scope rides in a signed token bound to the session.
3	Compute in code, narrate in the model	Invoice diffing became a full outer join on a stable line key. Arithmetic over a revenue document is where a fluent wrong answer is most expensive and least detectable.
3	Seed data engineered to a three-cause answer	A single-cause demo proves nothing a dashboard filter could not do. The ranked decomposition is a property of the fixture, not the model.
4	Enforcement scaffolding before the tools	A scoped query builder, per-call token validation and the audit chain are the parts that cannot be retrofitted.
5	The model's output is validated before disclosure	Prompt rules are a request. Citations must resolve, figures must appear in the evidence, and on failure the narration is withheld in favour of the computed ranking.
5	The agent carries the caller's token, not a service credential	An agent with broader authority than its caller is how this class of system becomes a confused deputy.
deploy	The demo states its own limits on every screen	The deployed build says, above the fold, that the data is synthetic and that the role selector <i>simulates</i> an identity rather than authenticating one. A reviewer finds that gap in ninety seconds either way; finding it announced reads as judgement, finding it hidden reads as a claim that did not survive.
deploy	A rate limiter before the URL is public	A public demo spends real inference quota on my account. A Durable Object holds a fixed hourly window, because an isolate-local counter is a rate limit in appearance only — Workers are not one process.

Design prompts

Prompt 1 — the shape

Thu 27 Aug

PROMPT

fast-track assignment: an MCP server on the Agents SDK, rather than another chat UI.

Can you please give me 4 line summary of what problem this will solve?

OUTCOME

Confirmed the framing and surfaced the follow-on question the shape forces and a chat UI lets you skip — *what is an agent allowed to read versus change, and how is that audited* — which became Prompt 2.

Prompt 2 — the tool surface

Fri 28 Aug

PROMPT

Spec the tool surface before we write any code.

Context: a billing-explainability system over a usage-based billing model — subscriptions, metered usage, invoices, and a ledger. Three components: (1) an MCP server on the Cloudflare Agents SDK exposing the billing systems as tools; (2) a Workers AI agent that connects to it as an MCP client and answers questions like "why did this account's invoice go up 40% in July?"; (3) a chat surface over the agent. Durable Objects hold session state and the audit trail.

Give me:

1. The tool list. For each: name, parameters with types, return shape, and one line on what a model would use it for. Classify each as READ or WRITE. I expect the read surface to be much wider than the write surface — tell me if you disagree, and why.
2. The authorization boundary. Who is the caller, and how does that identity travel from the chat session, through the agent, to the MCP server, and into the data? Be specific about where the check happens, not just that one happens. Assume two caller types: an internal support engineer, and an account-scoped customer.
3. Write gating. For every WRITE tool, say whether a model may call it unattended or whether it requires human confirmation — and show me the mechanism, not the intention. If that means a propose/confirm tool pair, or MCP elicitation, spec it.
4. The audit record. What is written on every tool call, in what shape, and what makes it trustworthy after the fact.
5. The negative space. Name three things that could plausibly be tools here and should not be, with the reason for each.

Then mark the minimum subset that must exist for a working two-day demo, versus what we specify and document but do not build. Do not write code yet.

OUTCOME

The full specification. Three results changed the build: invoice diffing became a deterministic tool rather than a model task; *apply_credit* was removed from the surface entirely in favour of propose → out-of-band approval → server applies; and the rule that **account_id in tool arguments is a selector, not an authorization claim**. Also the audit design — append-only, hash-chained, as-of pinned, denials first-class — and a two-day subset of seven tools with both roles real.

Prompt 3 — schema and fixture

Fri 28 Aug

PROMPT

Now the data model and the seed data.

Build the D1 schema behind the seven tools in the build subset: accounts, subscriptions, subscription_change_events, rate_cards, usage_records, invoices, invoice_line_items, credit_proposals, and the audit log's own storage shape.

Constraints:

- Every table holding pricing or entitlement data must be temporal – effective_from / effective_to – because as_of is a required parameter on get_rate_card and the audit record pins a data version. Do not give me a schema that can only represent the present.
- invoice_line_items must carry enough structure for compare_invoices to produce a ranked delta without heuristics: a stable line key that survives across periods, the metric, quantity, unit price, and whether the line is prorated.
- The audit log is append-only with a prev_hash column. Show me the write path, and convince me there is no update path.

Then the seed data. Two accounts, three months. The July invoice on account 1 must be up ~40%, and that increase must decompose into exactly three causes of different magnitudes: a usage step change (largest), a mid-period plan change producing a prorated line (middle), and an expired promotional discount (smallest). Give me the actual numbers and show me the arithmetic that produces the 40%, so I can verify the demo answers correctly before any agent code exists.

Finally: flag anything in this schema that would let a customer-scoped read accidentally return another account's data.

OUTCOME

A stable *line_key* with *UNIQUE (invoice_id, line_key)*, reducing *compare_invoices* to a full outer join so an expired discount is a NULL on one side rather than a model inference; *account_id* denormalised onto every tool-reachable child table so the scope predicate sits in the same WHERE clause as the selector. Fixture verified: June \$678.00 → July \$949.16, **+\$271.16 = 39.994%**, decomposing exactly into \$136.00 / \$85.16 / \$50.00. Six tenancy-leak vectors ranked.

Prompt 4 — enforcement before tools

Fri 28 Aug

PROMPT

Now build the MCP server. Cloudflare Workers, the Agents SDK's McpAgent, TypeScript, D1 for the billing tables, a separate Durable Object for the audit log.

Build the enforcement scaffolding first – I want it correct before the tools get interesting:

1. A ScopedDb wrapper that is the ONLY route from a tool handler to D1. It takes the principal at construction and injects the account predicate into every query. Tool handlers must not be able to issue raw SQL – show me what happens structurally if one tries, not just that we would catch it in review.
2. Auth middleware that validates the token on every tools/call, not only at connect, and derives account scope from the token and never from tool arguments. A call whose requested account_id falls outside scope fails, and the denial is written to the audit log with requested_scope recorded.
3. Role-scoped tools/list – a customer session must not see propose_credit in its manifest at all.

4. The audit D0: `append()` and `read()` only, the hash chain, and a `verify()` that walks from genesis. The entry is written BEFORE the tool response is returned.

5. Then the tools, `compare_invoices` first – full outer join on `line_key` producing a delta ranked by absolute contribution. No description matching anywhere.

Include one test I care about more than the others: a customer-scoped session for Northwind requests a Harborlight invoice id, is denied, and the denial appears in the audit log. Write it so it fails loudly if someone later refactors the scope predicate out of the query builder.

OUTCOME

The enforcement answer came out stronger than "we would catch it in review": the D1 binding sits in an **ECMAScript #private field** — a TypeScript *private* is erased at compile time and defeated by a cast — and *ScopedDb* exposes named repository methods rather than a query interface. *env.DB* is referenced in exactly one module and a test scans the source tree to assert it. Token re-verified on every call; the audit wrapper fails closed.

Prompt 5 — the agent, and the check on it

Fri 28 Aug

PROMPT

Now the agent and the chat surface – the third component.

Build a Workers AI agent that connects to the MCP server as a CLIENT and answers the billing question end to end, plus a chat page over it.

1. The agent is a Durable Object holding session state: the principal bound at construction and immutable, the conversation, and the tool-call trace. It connects to /mcp with the session's own token, so it inherits the caller's scope and cannot exceed it.

2. Use a Workflow for the investigation, because it is genuinely multi-step and I want the steps durable and individually retryable: resolve account, find the two invoices, compare them, then for each contributing line above a threshold pull the supporting evidence, then compose.

3. The composition prompt is the only real prompt engineering in this build and I want it written defensively. The model receives the computed delta and the evidence. It must NOT do arithmetic, must NOT name a cause that is not in the ranked list, and must cite a `line_key` or a usage date for every claim it makes. Show me what it does when reconciled is false – I want it to say the evidence does not add up, not fill the gap.

4. The chat page: plain HTML, streams the answer, and renders the tool-call trace beside it so a reviewer can see which tools ran, in what order, and what each returned. The trace is the demo.

5. Add a role switch in the UI, support engineer versus customer, so the same question against the same data produces a different manifest and a different answer.

OUTCOME

The composition prompt, plus a **validator that checks the model obeyed it** — citations must resolve, figures must appear verbatim in the evidence, an unreconciled comparison must be disclosed, no commitments. On failure the narration is

withheld and the computed ranking shown. Verified against five adversarial cases. Also: the agent carries the caller's own token rather than a service credential.

Prompt 6 — audit the artifact's own claims

Sat 29 Aug

PROMPT

Check that nothing in the repo claims a capability it does not have. Grep the README and the docs against what the code actually does and tell me every place they disagree. I would rather ship a smaller honest artifact than a larger one that overstates.

Then the honest gap list for the README — everything specified and not built, everything stubbed, and anything that would not survive production, written the way a reviewer would find it rather than the way a candidate would present it.

OUTCOME

Produced the "what is deliberately not built" section and the three-item "what I'd do next", each with the reason it is next. Also caught a duplicate: the audit DDL was in both the D1 schema and the Durable Object, so *db:init* was creating a second, empty, never-written *audit_log* in D1 — a table that would read as an audit log to anyone who found it. Removed, with a structural test asserting it stays out.

What running it exposed

Six defect classes, none of which were visible from reading the code. Every one was found by executing something — the suite, the server, or the model.

1 • The test suite would not start, and one test would have passed for the wrong reason

Symptom: the pool refused to start; then a misleading `this.snapshotClient.startCurrentRun` is not a function.

Causes: Workflows cannot run with per-test `isolatedStorage`; vitest 2.1 was outside the Workers pool's supported peer range; the structural tests used `node:fs` inside `workerd`, where there is no filesystem; the setup file **never loaded the seed fixture**; and its SQL splitter counted `BEGIN TRANSACTION` as a block opener, concatenating all 409 statements into one and failing with `SQLITE_TOOBIG`.

Fix: two vitest projects (`workerd` for behaviour, `node` for source-scanning), pinned vitest, fixture SQL inlined via Vite `?raw` imports, splitter corrected, and the loader made to fail loudly rather than swallow errors with `.catch(() => {})`.

Lesson. The fixture-never-loaded defect is the one that matters. The cross-tenant denial test would have reported *pass* against an empty database — asserting that no data leaked when there was no data. A test that passes for the wrong reason is worse than one that fails, because it retires the question.

2 • The MCP SDK cannot load in the Workers test runtime

Symptom: `SyntaxError: Unexpected token ':'` pointing into `ajv/dist/core.js`.

Cause: ajv's CJS build does `require('./refs/data.json')`. The Workers test runtime parses that JSON as JavaScript and dies on the first colon. A harness problem, not a runtime one — esbuild handles it under `wrangler dev`.

Fix: rather than stub the dependency, the enforcement path was **extracted out of the MCP agent into `guard.ts`**, and the server reduced to a thin adapter over it.

Lesson. The harness failure was the proximate cause; the reason the change stayed is that it is correct independently. A security boundary that can only be exercised by booting a protocol server is one that, in practice, does not get exercised.

3 • The application returned HTML where the client expected JSON

Symptom: `Unexpected token '<', "<!DOCTYPE"... is not valid JSON` in the browser.

Causes, four: the `agents` package dynamically imports `ai`, which was not a dependency, so the build failed outright; `ctx` was spread into a plain object, losing `waitUntil` and the props channel; the MCP client **never performed the `initialize` handshake**, so every call was rejected before reaching a tool; and `McpAgent.serve` defaulted to a binding name that did not exist in the config.

Fix: dependency added, props attached to the real `ExecutionContext`, full Streamable-HTTP handshake with session id, explicit binding name.

Lesson. The browser's message described the *shape* of the failure, not its origin. Wrapping every API route so errors return `{error}` as JSON turned each subsequent bug into one readable line, and was worth more than any individual fix.

4 • The Worker hung — and so did every refusal

Symptom: *"The Workers runtime canceled this request because it detected that your Worker's code had hung and would never generate a response."*

Cause: the agent Durable Object awaited the Workflow to completion inside the request handler, sleeping in a loop.

Fix: `start()` returns an instance id immediately; the client polls `/api/status`.

The related defect underneath it: Workflows retries a failed step by default, so a scope refusal sat in `running` for minutes, re-deciding something that will never change and writing a fresh denial into the audit log on every attempt. And a `NonRetryableError` thrown inside `step.do` is fatal *at the step boundary* — it never reaches a `try/catch` around `run()`, so the user saw "a step threw an error". A refusal was returned as a **value** instead.

Lesson. "A denial is an outcome, not a crash" turned out to be a design rule, not an error-handling detail. Access control that reports itself as a fault teaches the reader that the system is broken rather than that it is working.

5 • One error message, four causes, and the fatal one was a CLI flag

Symptom: Binding AI needs to be run remotely on every request, unchanged by repeated config edits.

Causes, peeled in order: the AI binding needs `remote: true`; the account had no **workers.dev** subdomain, without which the remote-binding proxy cannot be established; the default model id did not exist on the account (`llama-3.3-70b-instruct` vs the available `-fp8-fast`); and underneath all of it the `dev` script carried `--local`, which disables remote bindings entirely and would have produced the identical error after every other fix was correct.

Fix: plain `wrangler dev`, subdomain registered, model id corrected, and an `AI_MODEL` override so an unavailable model is a config change rather than a code edit.

Lesson. I read the message as a configuration problem for several rounds because that is what it sounds like, and never checked what command was actually being run. What broke it open was running the same thing a different way — `--remote` — and reading how it failed *differently*. A diagnostic that changes the failure mode is worth more than one that confirms it.

6 • The validator was decorative until the model was genuinely wrong

Symptom: the first real narrations **passed validation while being wrong**. One reported *"increased by 27,116 cents"* with no bracket citations at all. Another read the included allowance as a previous value — *"decreased by 12 requests (from 112 to 100)"* — and reported *"a reduction of \$400"* where the figure was \$4.00.

Causes: the money check required decimals, so `$400` and `$120` were never examined; zero citations produced zero *invalid* citations, so the citation rule enforced nothing; and the evidence handed the model raw cents alongside a field named `included_qty` sitting next to `quantity`, which invites a before/after reading.

Fix: optional-decimal money matching, a minimum citation count tied to the number of material causes, an explicit raw-cents check, and an evidence bundle that presents money as formatted dollar strings with unambiguous field names (`metered_quantity`, `included_allowance`). Re-verified against the two real failures plus a compliant answer.

Lesson. A validator is only tested by output that is actually wrong. Five synthetic adversarial cases passed from the day it was written; the first real model output defeated it in two different ways within a minute. And half the fix belonged to the *evidence*, not the model — a field name that invites a misreading is a defect in the prompt, not a failure of reasoning.

What deploying it exposed

Everything above was found on a laptop. These three were found in the twenty minutes between "it works" and "somebody else can open it", and none of them were visible from the local build.

7 • Local D1 and remote D1 are not the same database

Symptom: the fixture applied cleanly to local D1 for three days, then was rejected outright by remote D1 on the first attempt.

Cause: the generated seed wrapped its 407 inserts in `BEGIN TRANSACTION / COMMIT`. Miniflare's local D1 accepts that; the managed service owns its own transaction boundaries and refuses an explicit one.

Fix: dropped the wrapper and let the service batch.

Lesson. "Works locally" for a storage binding means "works against the emulator", and the emulator is the more permissive of the two. The asymmetry runs the other way for Workers AI, which has no emulator at all — so on this platform *neither* environment is a superset of the other, and each binding has to be exercised where it will actually run before a URL is shared.

8 • A half-applied deploy was a state no script could move forward

Symptom: `table accounts already exists`, and on the retry after that, `index ux_rate_cards_open already exists`.

Cause: the init and seed scripts were written for a clean local database — bare `CREATE TABLE` and `INSERT INTO`. The first remote run failed partway through, which left the database neither empty nor complete, and there was no command that could take it the rest of the way.

Fix: `CREATE TABLE / INDEX IF NOT EXISTS`, `INSERT OR REPLACE`, and explicit `:remote` script variants so the flag is not a thing to remember at the wrong moment.

Lesson. A step that only runs against a clean slate is not a deploy step, it is a one-shot — and the failure that matters is not the first error, it is having no way to retry after it. Worth recording separately: my own patch matched `CREATE TABLE` and `CREATE INDEX` and silently missed the single `CREATE UNIQUE INDEX`. A pattern-based edit that looks exhaustive and is not, caught only because the next run failed on that exact line.

9 • The documentation referenced three screenshots that did not exist

Symptom: none locally, which is the point — the repo had not been pushed, and nothing in the build checks a markdown link.

Cause: the walkthrough was written before the screenshots were taken and named five files. Three were never captured. On GitHub the README would have opened with a broken-image icon.

Fix: rewrote those sections as prose describing what the screenshots that *do* exist actually show, and added a check that walks every relative link in every markdown file in the repo.

Lesson. The README is read before any code is, and a broken image at the top of it is a competence signal that arrives before the argument does. A document that names its own artifacts needs the same kind of tripwire the source tree already has — the structural tests in this repo exist because an intention to remember is not a control, and documentation is not exempt from that.

On method

The prompts that produced the most durable work were the ones that specified a **constraint and a refusal** rather than a feature: *show me what happens structurally if a handler tries raw SQL*; *name three things that could be tools and should not be*; *show me the mechanism, not the intention*. Each of those produced a decision that survived contact with the running system.

The other pattern worth naming: every defect above was found by **executing** something. None were visible from reading. Two of them — the fixture that never loaded, and the validator that passed wrong answers — were cases where the code reported success while doing nothing, which is the failure mode that reading is least able to catch and the reason the structural tests in this repo scan the source tree rather than only exercising behaviour.

The deployment section extends that pattern rather than adding a new one. All three of those defects were invisible on a working laptop and appeared the moment the work had to survive being handed to someone else —

which is the same relationship between *it runs* and *it holds* that the audit chain, the output validator and the structural tests were each built to enforce inside the system.